

# OP\_ML User Guide

## Order Parameter Computation & ML Phase Classification

This tool computes structural order parameters from LAMMPS trajectory files and trains machine learning models for phase classification. It uses a fast Fortran backend with MPI parallelism.

## 1. Installation

Run these commands in your terminal:

```
pip install git+https://github.com/dikshaiisc/OrderParameter.git
```

Then build the Fortran extension (needed for OP computation):

```
cd $(python3 -c "import op_ml,os; print(os.path.dirname(op_ml.__file__)))/fortran
make
```

**Note:** If make fails, the tool will try to build automatically when you first run `OP_ML compute`. You need a Fortran compiler (`mpiifx` or `gfortran`) and `meson` installed.

## 2. What This Tool Does

- **Computes 383 order parameters** from LAMMPS trajectory files
- **Handles any system** — water, ice, CaCO<sub>3</sub>, polymers, metals, etc.
- **Selects atoms by LAMMPS type number** (not element name)
- **Lets you choose neighbor types** — e.g., compute OPs for Ca considering Ca+C as neighbors
- **Lets you choose which OPs to compute** — all, or specific categories (Q, W, B, etc.)
- **Processes only the last N frames** if you don't need the full trajectory
- **Runs in parallel** using MPI for fast computation
- **Validates data** — stops immediately if NaN values are found (never drops data)
- **Trains XGBoost ML models** with automatic feature selection
- **Detects overfitting** and selects the best model
- **Generates plots** — histograms, scatter plots, confusion matrices

## 3. Commands

The tool has 4 commands. Type `OP_ML --help` to see them all.

| Command        | What It Does                                   |
|----------------|------------------------------------------------|
| OP_ML compute  | Compute order parameters from trajectory files |
| OP_ML train    | Train ML model on an existing OP CSV file      |
| OP_ML run      | Do both: compute OPs then train ML             |
| OP_ML validate | Check a CSV file for NaN values                |

## 4. All Options

### 4.1 OP Computation Options

| Option                | Default      | What It Does                                               |
|-----------------------|--------------|------------------------------------------------------------|
| --traj DIR            | (required)   | Directory with .lammprj files. Each filename = phase label |
| --central-type INT    | (asked)      | LAMMPS atom type for the central atom (e.g., 1 for Ca)     |
| --neighbor-types LIST | central only | Which atom types to use as neighbors (e.g., 1,2)           |
| --rcut FLOAT          | 4.5          | Neighbor cutoff distance in Angstroms                      |
| --op-type             | avg          | Which OPs to output: avg, local, or both                   |
| --op-categories       | all          | Which OP types: B,D,F,I,Q,W,LQ,LW or all                   |
| --last-frames N       | all          | Process only the last N frames                             |

## 4.2 ML Training Options

| Option              | Default    | What It Does                                                    |
|---------------------|------------|-----------------------------------------------------------------|
| --data CSV          | (required) | Input CSV file with OP columns and a Class column               |
| --max-sfs INT       | 6          | Maximum number of features to try in selection                  |
| --overfit-gap FLOAT | 0.05       | If train-test gap exceeds this, model is flagged as overfitting |
| --n-iter INT        | 25         | Number of hyperparameter combinations to try                    |
| --param-grid JSON   | built-in   | Your own hyperparameter grid (JSON file)                        |
| --no-interactive    | off        | Skip all prompts, use default values                            |

## 4.3 Output Options

| Option       | Default | What It Does                   |
|--------------|---------|--------------------------------|
| --outdir DIR | results | Where to save all output files |

## 5. How Atom Selection Works

Atoms are selected by their **LAMMPS type number** (the integer in column 3 of the trajectory file), not by element name. When you run the tool, it reads the trajectory and shows you:

```
Atom types found in trajectory:
  Type 1 = Ca
  Type 2 = O
  Type 3 = C
Central atom type (integer) [default=1]:
```

You pick the type number. Examples:

| System             | Central | Neighbors   | Command                                 |
|--------------------|---------|-------------|-----------------------------------------|
| Water (O-O)        | 1 (O)   | 1 (O only)  | --central-type 1                        |
| Water (O with all) | 1 (O)   | 1,2 (O+H)   | --central-type 1 --neighbor-types 1,2   |
| CaCO3 (Ca-Ca)      | 1 (Ca)  | 1 (Ca only) | --central-type 1                        |
| CaCO3 (Ca-Ca+C)    | 1 (Ca)  | 1,3 (Ca+C)  | --central-type 1 --neighbor-types 1,3   |
| CaCO3 (Ca-all)     | 1 (Ca)  | 1,2,3 (all) | --central-type 1 --neighbor-types 1,2,3 |

**Note:** If you don't provide --central-type, the tool auto-detects the first type from the trajectory. In interactive mode (no MPI), it asks you to confirm.

## 6. Choosing Which Order Parameters to Compute

By default, all 383 OPs are computed. You can select specific categories:

| Category | Count | What It Is                      |
|----------|-------|---------------------------------|
| B        | 42    | Bond-angle order parameters     |
| D        | 25    | Distance-triplet parameters     |
| F        | 275   | Mixed distance-angle parameters |
| I        | 1     | Jiang tetrahedral invariant     |
| Q        | 10    | Steinhardt Q parameters         |
| W        | 10    | Steinhardt W parameters         |
| LQ       | 10    | Local alignment Q               |
| LW       | 10    | Local alignment W               |
| all      | 383   | Everything (default)            |

Examples:

```
# Only Steinhardt parameters:
OP_ML compute --traj phases/ --central-type 1 --op-categories Q,W,LQ,LW

# Only bond-angle:
OP_ML compute --traj phases/ --central-type 1 --op-categories B

# Steinhardt + tetrahedral:
OP_ML compute --traj phases/ --central-type 1 --op-categories Q,W,I
```

## 7. Processing Only the Last N Frames

If your trajectory has 2000 frames but you only need the last 10 (e.g., the equilibrated portion):

```
OP_ML compute --traj phases/ --central-type 1 --last-frames 10
```

The output will show:

```
[OP_ML] last 10 of 2000 frames, 1000 central atoms, 1 rank(s)
```

## 8. NaN Handling

If any NaN (Not a Number) value is found in the OP data, the tool **stops immediately** and tells you exactly which columns have the problem:

```
ERROR: NaN values detected in OP data!
Columns with NaN values:
  Q_6avg: 12 NaN values

The pipeline CANNOT proceed with NaN values.
Please check your input file and fix the issue.
```

The tool never drops or removes NaN data. You must fix the input.

## 9. Phase Clubbing (Merging Phases)

When training ML, the tool asks if you want to merge some phases together. For example, you might want to combine two similar ice phases into one class:

```
Phases found: [1] aragonite [2] calcite [3] liquid

Your choice [1/2]: 2
Phases to merge: 1,2
Group name: Crystal
  Merged ['aragonite','calcite'] -> 'Crystal'
Phases to merge: done

Final: aragonite->Crystal, calcite->Crystal, liquid->liquid
```

Use `--no-interactive` to skip this and keep all phases separate.

## 10. Running in Parallel (MPI)

For faster OP computation, use MPI. Each process handles different frames:

```
# Use 4 processes:
mpiexec -n 4 OP_ML compute --traj phases/ --central-type 1 --rcut 5.0

# Use 8 processes:
mpiexec -n 8 OP_ML compute --traj phases/ --central-type 1 --rcut 5.0
```

| Processes | Speedup | Time (1000 frames, 1000 atoms) |
|-----------|---------|--------------------------------|
| 1         | 1x      | ~45 min                        |
| 4         | ~4x     | ~12 min                        |
| 8         | ~8x     | ~6 min                         |
| 16        | ~16x    | ~3 min                         |

**Note:** With MPI, interactive prompts are disabled. Provide all options via flags (`--central-type`, `--rcut`, etc.).

## 11. Output Files

Everything goes into the `--outdir` directory:

| File                                       | What It Contains                                    |
|--------------------------------------------|-----------------------------------------------------|
| OP.csv                                     | Order parameters for all phases with a Class column |
| ml_results/model_summary.csv               | Summary: features, accuracy, overfitting status     |
| ml_results/xgb_1feat.pkl ... xgb_6feat.pkl | Trained ML models (one per feature count)           |
| ml_results/feature_importances.png         | Which OPs are most important                        |
| ml_results/accuracy_curve.png              | How accuracy changes with more features             |
| ml_results/hist_1feat.png                  | Histogram for 1-feature model                       |
| ml_results/scatter_2feat.png               | Scatter plot for 2-feature model                    |
| ml_results/scatter_3feat.png               | 3D scatter for 3-feature model                      |
| ml_results/cm_kfeat.png                    | Confusion matrix for each model                     |

## 12. Custom Hyperparameters

Create a JSON file with your own XGBoost parameter grid:

```
# my_params.json
{
  "n_estimators": [100, 500, 1000],
  "learning_rate": [0.01, 0.05, 0.1],
  "max_depth": [3, 5, 10],
  "subsample": [0.5, 0.7, 0.9]
}
```

Then:

```
OP_ML train --data OP.csv --param-grid my_params.json --n-iter 50
```

## 13. Windows and macOS

| Platform | OP Computation                  | ML Training |
|----------|---------------------------------|-------------|
| Linux    | Yes (native)                    | Yes         |
| macOS    | Yes (brew install gcc open-mpi) | Yes         |
| Windows  | Via WSL only                    | Yes         |

On Windows/Mac without a Fortran compiler, compute OPs on a Linux cluster, copy the CSV file, and run ML locally:

```
# On Linux:
OP_ML compute --traj phases/ --central-type 1 --outdir results/
# Copy results/OP.csv to your laptop

# On Windows/Mac:
OP_ML train --data OP.csv --outdir results/
```

## 14. Working Example: Water/Ice Classification

This example shows the complete workflow for classifying water/ice phases.

### Step 1: Prepare trajectory files

Name each file by its phase and put them in one folder:

```
phases/  
Ice1c.lampstrj  
Ice1h.lampstrj  
Liquid.lampstrj  
S1.lampstrj  
S2.lampstrj  
SH.lampstrj
```

### Step 2: Compute order parameters

```
# Water: type 1 = O, type 2 = H  
# Compute 0-0 order parameters for last 50 frames  
mpiexec -n 8 OP_ML compute --traj phases/ --central-type 1 --rcut 4.5 \  
    --last-frames 50 --outdir results/
```

### Step 3: Train ML model

```
OP_ML train --data results/OP.csv --outdir results/
```

The tool will ask about phase clubbing and ML parameters.

### Step 4: Check results

```
cat results/ml_results/model_summary.csv
```

Look for the model with highest test accuracy and Status = OK (not OVERFIT).

### Step 5: Use the model

```
import pickle  
model = pickle.load(open('results/ml_results/xgb_3feat.pkl', 'rb'))  
predictions = model.predict(X_new)
```

## 15. Working Example: CaCO<sub>3</sub> Phase Classification

This example shows an atomic system (no molecules — each atom is independent).

### Step 1: Prepare trajectory files

```
phases/  
aragonite.lampstrj    (Type 1=Ca, Type 2=O, Type 3=C)  
calcite.lampstrj
```

### Step 2: Compute order parameters

```
# Ca as central, Ca+C as neighbors, cutoff 6 Å, last 10 frames  
OP_ML compute --traj phases/ --central-type 1 --neighbor-types 1,3 \  
    --rcut 6.0 --last-frames 10 --outdir results/  
  
# Or only Steinhardt Q parameters:  
OP_ML compute --traj phases/ --central-type 1 --neighbor-types 1,3 \  
    --rcut 6.0 --last-frames 10 --op-categories Q --outdir results/
```

### Step 3: Train and evaluate

```
OP_ML train --data results/OP.csv --outdir results/
```

## 16. Full Pipeline in One Command

Use `OP_ML run` to do everything at once:

```
# Water/Ice:
OP_ML run --traj phases/ --central-type 1 --rcut 4.5 --outdir results/

# CaCO3 with MPI:
mpiexec -n 4 OP_ML run --traj phases/ --central-type 1 \
  --neighbor-types 1,3 --rcut 6.0 --last-frames 10 --outdir results/

# Everything specified (no prompts):
mpiexec -n 8 OP_ML run --traj phases/ --central-type 1 \
  --neighbor-types 1,3 --rcut 6.0 --last-frames 50 \
  --op-categories Q,W --max-sfs 4 --overfit-gap 0.03 \
  --no-interactive --outdir results/
```

## 17. Troubleshooting

| Problem                     | Solution                                                                                      |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| Fortran extension not built | Run <code>make</code> in the <code>fortran/</code> directory, or the tool tries to auto-build |
| Segfault with MPI           | Use fewer ranks (try <code>-n 2</code> or <code>-n 1</code> first)                            |
| NaN in output               | Check your trajectory — isolated atoms (zero neighbors) cause NaN                             |
| Wrong atom detected         | Use <code>--central-type</code> to specify the correct LAMMPS type number                     |
| Triclinic box error         | Supported since v1.1 — make sure you have the latest version                                  |
| NumPy version error         | Run: <code>pip install --upgrade numpy pandas pyarrow</code>                                  |

**Need help?** Run `OP_ML --help` to see all options.